



Courses : Advanced Web Programming
Courses : D4 – Informatics Engineering / D4 – Business Information Systems
Semester : 4 (four) / 5 (five)
The meeting : 4 (four)

JOBSHEET 04 MODEL AND ELECTROQUENT ORM

We've talked about *Migration*, *Seeder*, *DB Façade*, *Query Builder*, and a little bit about *the Eloquent ORM* in Laravel. Before we enter the creation of a website-based application, it would be good if we need to prepare a database as a place to store the data on our application later. In addition, generally we also need to prepare the initial data that we use before creating an application, such as user administrator data, system setting data, etc.

In this meeting, we will understand how to display data, change data, and delete data using the Eloquent technique.

In accordance with **Case Study** PWL.pdf.

So our Laravel 10 project is still the same using **the PWL_POS repository**.

We will **use PWL_POS** project until the 12th meeting, as a project that we will learn

ORM (Object Relation Mapping) is a technique that turns a table into an object that is easy to use. The object created has the same properties as the fields — the fields in the table. The ORM serves as a liaison and at the same time makes it easier for us to create applications that use relational databases to make our tasks more efficient.

Pros - Advantages of Using ORM

1. There are many features such as transactions, connection pooling, migrations, seeds, streams, and so on.
2. Query commands have better performance, than we write them manually.
3. We write the data model in just one place, making it easier to update, maintain, and reuse the code.
4. Allows us to make good use of OOP (object oriented programming)



In Laravel itself, Eloquent ORM has been provided to make it easier for us to perform various kinds of queries to the database, and make our work easier because there is no need to write long sql queries to process data.

A. Properties of `$fillable` and `$guarded`

1. `$fillable`

Variables `$fillable` useful for registering attributes (column names) that we can fill in when inserting or updating to the database. Previously we understood adding a new record to the database. For the step of adding a `Variable $fillable` can be done by adding a *script* like below to the model file

```
protected $fillable = ['level_id', 'username'];
```

Practicum 1 - `$fillable`:

1. Open the model file with the name `UserModel.php` and add `$fillable` as shown below

```
class UserModel extends Model
{
    use HasFactory;

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
    /**
     * The attributes that are mass assignable.
     *
     * @var array
     */
    protected $fillable = ['level_id', 'username', 'nama', 'password'];
}
```

2. Open the controller file with the name `UserController.php` and change the script to add the new data as shown below



```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\UserModel;
7  use Illuminate\Support\Facades\Hash;
8
9  class UserController extends Controller
10 {
11     public function index()
12     {
13         $data = [
14             'level_id' => 2,
15             'username' => 'manager_dua',
16             'nama' => 'Manager 2',
17             'password' => Hash::make('12345')
18         ];
19         UserModel::create($data);
20
21         $user = UserModel::all();
22         return view('user', ['data' => $user]);
23     }
24 }
25
```

3. Save the program code from Steps 1 and 2, and run the web server. Then run the localhostPWL_POS/public/user link in the browser and observe what happens
4. Modify the `UserModel.php` model file as shown in the image below in section `$fillable`

```
protected $fillable = ['level_id', 'username', 'nama'];
```

5. Change back the controller file `UserController.php` as in the image below only the array part on the `$data`

```
public function index()
{
    $data = [
        'level_id' => 2,
        'username' => 'manager_tiga',
        'nama' => 'Manager 3',
        'password' => Hash::make('12345')
    ];
    UserModel::create($data);

    $user = UserModel::all();
    return view('user', ['data' => $user]);
}
```

6. Save the program code Step 4 and 5. Then run it on the browser and observe what happens
7. Report the results of this Practicum-1 and *commit* the changes to *git*.

2. `$guarded`



The opposite of `$fillable` is `$guarded`. All the columns we add to the `$guarded` will be ignored by Eloquent when we do the insert/update. By default `$guarded` the contents of an `array("*")`, which means that all attributes cannot be set via **mass assignment**. **Mass Assignment** is a powerful feature that simplifies the process of setting multiple model attributes at once, saving time and effort. In this practicum, we will explore the concept of bulk assignment in Laravel and how it can be effectively leveraged to improve your development workflow.

B. Retrieving Single Models

In addition to retrieving all records that match a particular query, you can also retrieve a single record using the `find`, `first`, or `firstWhere` methods. Instead of returning a set of models, this method returns a single model instance and is done on the controller:

```
// Ambil model dengan kunci utamanya...
$user = UserModel::find(1);

// Ambil model pertama yang cocok dengan batasan kueri...
$user = UserModel::where('level_id', 1)->first();

// Alternatif untuk mengambil model pertama yang cocok dengan batasan kueri...
$user = UserModel::firstWhere('level_id', 1);
```

Practicum 2.1 – Retrieving Single Models

1. Open the controller file with the name `UserController.php` and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::find(1);
        return view('user', ['data' => $user]);
    }
}
```

2. Open the view file with the name `user.blade.php` and change the *script* as shown below



```
<body>
<h1>Data User</h1>
<table border="1" cellpadding="2" cellspacing="0">
  <tr>
    <td>ID</td>
    <td>Username</td>
    <td>Nama</td>
    <td>ID Level Pengguna</td>
  </tr>
  <tr>
    <td>{{ $data->user_id }}</td>
    <td>{{ $data->username }}</td>
    <td>{{ $data->nama }}</td>
    <td>{{ $data->level_id }}</td>
  </tr>
</table>
</body>
```

3. Save the program code from Steps 1 and 2. Then run it on the *browser* and observe what is happening and give an explanation in the report
4. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::where('level_id', 1)->first();
        return view('user', ['data' => $user]);
    }
}
```

5. Save the program code Step 4. Then run it on the *browser* and observe what is happening and give an explanation in the report
6. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstWhere('level_id', 1);
        return view('user', ['data' => $user]);
    }
}
```

7. Save the program code Step 6. Then run it on the *browser* and observe what is happening and give an explanation in the report

Sometimes you may want to take some other action if no results are found. The `findOr` and `firstOr` methods will return a single model instance or, if no results are found, it will run



inside the function. The value returned by the function will be considered as the result of this method:

```
$user = UserModel::findOr(1, function () {  
    // ...  
});  
  
$user = UserModel::where('level_id', '>', 3)->firstOr(function () {  
    // ...  
});
```

8. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller  
{  
    public function index()  
    {  
        $user = UserModel::findOr(1, ['username', 'nama'], function () {  
            abort(404);  
        });  
        return view('user', ['data' => $user]);  
    }  
}
```

9. Save the program code Step 8. Then on the *browser* and observe what happens and give an explanation in the report
10. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller  
{  
    public function index()  
    {  
        $user = UserModel::findOr(20, ['username', 'nama'], function () {  
            abort(404);  
        });  
        return view('user', ['data' => $user]);  
    }  
}
```

11. Save the program code Step 10. Then run it on the *browser* and observe what is happening and give an explanation in the report
12. Report the results of this Practicum-2.1 and *commit* the changes to *git*.

Practicum 2.2 – Not Found Exceptions

Sometimes you may want to provide an exception if the model is not found. This is very useful in *routes* or controllers. The `findOrFail` and `firstOrFail` methods will retrieve the



first result of the query; however, if no results are found, an `Illuminate\Database\Eloquent\ModelNotFoundException` will be thrown. Here are the steps below:

1. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::findOrFail(1);
        return view('user', ['data' => $user]);
    }
}
```

2. Save the program code Step 1. Then run it on the *browser* and observe what is happening and give an explanation in the report
3. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::where('username', 'manager9')->firstOrFail();
        return view('user', ['data' => $user]);
    }
}
```

4. Save the program code Step 3. Then run it on the *browser* and observe what is happening and give an explanation in the report
5. Report the results of this Practicum-2.2 and *commit* the changes to *git*.

Practicum 2.3 – Retrieving Aggregates

When interacting with the Eloquent model, you can also use the `count`, `sum`, `max`, and other aggregate methods provided by the Laravel query builder. As you might expect, this method returns a scalar value and an example of an Eloquent model:

```
$count = UserModel::where('active', 1)->count();
$max = UserModel::where('active', 1)->max('price');
```



1. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::where('level_id', 2)->count();
        dd($user);
        return view('user', ['data' => $user]);
    }
}
```

2. Save the program code Step 1. Then run it on the *browser* and observe what is happening and give an explanation in the report
3. Make the number of *scripts* in step 1 appear on the *browser* page, for example, you can see the image below and change *the script* in the view file so that the data can appear

Data User

Jumlah Pengguna
2

4. Report the results of this Practicum-2.3 and *commit* the changes to *git*.

Practicum 2.4 – *Retreiving or Creating Models*

The `firstOrCreate` method is a method to retrieve *data* based on the value you want to find, if the data is not found, this method will insert it into the database table according to the value entered.

The `firstOrCreate` method, such as `firstOrCreate`, will try to find/retrieve *records/data* in the database that match the given attributes. However, if no data is found, the data will be prepared for insertion into the database and the new model will be returned. Note that the model returned by `firstOrCreate` has not been saved to the database. You need to call the `save()` method manually to save it:



```
$user = UserModel::firstOrCreate([
    'username' => 'manager',
    'nama' => 'Manager',
]);

$user = UserModel::firstOrCreate([
    'username' => 'manager',
    'nama' => 'Manager',
]);
```

1. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate([
            'username' => 'manager',
            'nama' => 'Manager',
        ]);

        return view('user', ['data' => $user]);
    }
}
```

2. Change the view file back with the `user.blade.php` name and change the *script* as shown below

```
<body>
<h1>Data User</h1>
<table border="1" cellpadding="2" cellspacing="0">
    <tr>
        <td>ID</td>
        <td>Username</td>
        <td>Nama</td>
        <td>ID Level Pengguna</td>
    </tr>
    <tr>
        <td>{{ $data->user_id }}</td>
        <td>{{ $data->username }}</td>
        <td>{{ $data->nama }}</td>
        <td>{{ $data->level_id }}</td>
    </tr>
</table>
</body>
```



3. Save the program code from Steps 1 and 2. Then run it on the *browser* and observe what is happening and give an explanation in the report
4. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager22',
                'nama' => 'Manager Dua Dua',
                'password' => Hash::make('12345'),
                'level_id' => 2
            ],
        );

        return view('user', ['data' => $user]);
    }
}
```

5. Save the program code Step 4. Then run it on the *browser* and observe what is happening and also check *phpMyAdmin* on the `m_user` table and give an explanation in the report
6. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager',
                'nama' => 'Manager',
            ],
        );

        return view('user', ['data' => $user]);
    }
}
```

7. Save the program code Step 6. Then run it on the *browser* and observe what is happening and give an explanation in the report
8. Change the controller file with the `UserController.php` name and change the *script* as shown below



```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager33',
                'nama' => 'Manager Tiga Tiga',
                'password' => Hash::make('12345'),
                'level_id' => 2
            ],
        );
        return view('user', ['data' => $user]);
    }
}
```

9. Save the program code Step 8. Then run it on the *browser* and observe what is happening and also check *phpMyAdmin* on the *m_user* table and give an explanation in the report
10. Change the controller file with the *UserController.php* name and change the script like this

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager33',
                'nama' => 'Manager Tiga Tiga',
                'password' => Hash::make('12345'),
                'level_id' => 2
            ],
        );
        $user->save();
        return view('user', ['data' => $user]);
    }
}
```

11. Save the program code Step 9. Then run it on the *browser* and observe what is happening and also check *phpMyAdmin* on the *m_user* table and give an explanation in the report
12. Report the results of this Practicum-2.4 and *commit* the changes to *git*.

Practicum 2.5 – Attribute Changes

Eloquent provides *isDirty*, *isClean*, and *wasChanged* methods to check the internal state of your model and determine how its attributes have changed since the model was first fetched.

The *isDirty* method determines whether any of the model's attributes have been changed since the model was fetched. You can pass the name of a specific attribute or a set of



attributes to the `isDirty` method to determine if any of the attributes are "dirty". This `isClean` method will determine whether an attribute remains unchanged since the model was fetched. This method also accepts optional attribute arguments:

```
$user = UserModel::create([
    'username' => 'manager44',
    'nama' => 'Manager44',
    'password' => Hash::make('12345'),
    'level_id' => 2,
]);

$user->username = 'manager45';

$user->isDirty(); // true
$user->isDirty('username'); // true
$user->isDirty('nama'); // false
$user->isDirty(['nama', 'username']); // true

$user->isClean(); // false
$user->isClean('username'); // false
$user->isClean('nama'); // true
$user->isClean(['nama', 'username']); // false

$user->save();

$user->isDirty(); // false
$user->isClean(); // true
```

1. Change the controller file with the `UserController.php` name and change the *script* as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::create([
            'username' => 'manager55',
            'nama' => 'Manager55',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ]);

        $user->username = 'manager56';

        $user->isDirty(); // true
        $user->isDirty('username'); // true
        $user->isDirty('nama'); // false
        $user->isDirty(['nama', 'username']); // true

        $user->isClean(); // false
        $user->isClean('username'); // false
        $user->isClean('nama'); // true
        $user->isClean(['nama', 'username']); // false

        $user->save();

        $user->isDirty(); // false
        $user->isClean(); // true
        dd($user->isDirty());
    }
}
```

2. Save the program code Step 1. Then run it on the *browser* and observe what is happening and give an explanation in the report



This `wasChanged` method determines whether any attributes were changed when the model was last saved in the current request cycle. If needed, you can provide an attribute name to see if a particular attribute has been changed:

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::create([
            'username' => 'manager11',
            'nama' => 'Manager11',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ]);

        $user->username = 'manager12';

        $user->save();

        $user->wasChanged(); // true
        $user->wasChanged('username'); // true
        $user->wasChanged(['username', 'level_id']); // true
        $user->wasChanged('nama'); // false
        $user->wasChanged(['nama', 'username']); // true
    }
}
```

3. Change the controller file with the `UserController.php` name and change the script as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::create([
            'username' => 'manager11',
            'nama' => 'Manager11',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ]);

        $user->username = 'manager12';

        $user->save();

        $user->wasChanged(); // true
        $user->wasChanged('username'); // true
        $user->wasChanged(['username', 'level_id']); // true
        $user->wasChanged('nama'); // false
        dd($user->wasChanged(['nama', 'username'])); // true
    }
}
```

4. Save the program code Step 3. Then run it on the browser and observe what is happening and give an explanation in the report
5. Report the results of this Practicum-2.5 and *commit* the changes to *git*.

Practicum 2.6 – Create, Read, Update, Delete (CRUD)



As we already know, CRUD stands for *Create, Read, Update* and *Delete*. CRUD is a term for the process of processing data on a database, such as entering data into a database, displaying data from a database, editing data on a database and deleting data from a database. Follow the steps below to perform CRUD with Eloquent

1. Open the file view on [the user.blade.php](#) and make the script look like below

```
<body>
<h1>Data User</h1>
<a href="/user/tambah">+ Tambah User</a>
<table border="1" cellpadding="2" cellspacing="0">
  <tr>
    <td>ID</td>
    <td>Username</td>
    <td>Nama</td>
    <td>ID Level Pengguna</td>
    <td>Aksi</td>
  </tr>
  @foreach ($data as $d)
    <tr>
      <td>{{ $d->user_id }}</td>
      <td>{{ $d->username }}</td>
      <td>{{ $d->nama }}</td>
      <td>{{ $d->level_id }}</td>
      <td><a href="/user/ubah/{{ $d->user_id }}">Ubah</a> | <a href="/user/hapus/{{ $d->user_id }}">Hapus</a></td>
    </tr>
  @endforeach
</table>
</body>
```

2. Open the controller file on *UserController.php* and create the script to *read* it as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```

3. Save the program code from Steps 1 and 2. Then run it on the *browser* and observe what is happening and give an explanation in the report
4. The next step is to create or add user data by creating a new file in the view with the name [user_tambah.blade.php](#) and making the script look like below



```
<body>
  <h1>Form Tambah Data User</h1>
  <form method="post" action="/user/tambah_simpan">

    {{ csrf_field() }}

    <label>Username</label>
    <input type="text" name="username" placeholder="Masukan Username">
    <br>
    <label>Nama</label>
    <input type="text" name="nama" placeholder="Masukan Nama">
    <br>
    <label>Password</label>
    <input type="password" name="password" placeholder="Masukan Password">
    <br>
    <label>Level ID</label>
    <input type="number" name="level_id" placeholder="Masukan ID Level">
    <br><br>
    <input type="submit" class="btn btn-success" value="Simpan">

  </form>
</body>
```

5. Add the script to the routes with the file name `web.php`. Add as below image

```
Route::get('/user/tambah', [UserController::class, 'tambah']);
```

6. Add a script to the controller with the file name `UserController.php`. Add the script in the class and create a new method with the plus name and put it under the method index as shown below

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }

    public function tambah()
    {
        return view('user_tambah');
    }
}
```

7. Save the program code Step 4 through 6. Then run it on the browser and click the "+ Add User" link, observe what is happening and give an explanation in the report
8. Add the script to the routes with the file name `web.php`. Add as below image

```
Route::post('/user/tambah_simpan', [UserController::class, 'tambah_simpan']);
```

9. Add a script to the controller with the file name `UserController.php`. Add the script in the class and create a new method with the name `tambah_simpan` and put it under the add method as shown below



```
public function tambah_simpan(Request $request)
{
    UserModel::create([
        'username' => $request->username,
        'nama' => $request->nama,
        'password' => Hash::make('$request->password'),
        'level_id' => $request->level_id
    ]);

    return redirect('/user');
}
```

10. Save the program code Step 8 and 9. Then run the link <localhost:8000/user/add> or localhost/PWL_POS/public/user/add in the browser and enter the form and save it, then observe what happens and give an explanation in the report
11. The next step is to *update* or change user data by creating a new file in the view with the name `user_ubah.blade.php` and making the script look like below

```
<body>
<h1>Form Ubah Data User</h1>
<a href="/user">Kembali</a>
<br><br>

<form method="post" action="/user/ubah_simpan/{{ $data->user_id }}">
    {{ csrf_field() }}
    {{ method_field('PUT') }}

    <label>Username</label>
    <input type="text" name="username" placeholder="Masukan Username" value="{{ $data->username }}">
    <br>
    <label>Nama</label>
    <input type="text" name="nama" placeholder="Masukan Nama" value="{{ $data->username }}">
    <br>
    <label>Password</label>
    <input type="password" name="password" placeholder="Masukan Password" value="{{ $data->password }}">
    <br>
    <label>Level ID</label>
    <input type="number" name="level_id" placeholder="Masukan ID Level" value="{{ $data->level_id }}">
    <br><br>
    <input type="submit" class="btn btn-success" value="Ubah">
</form>
</body>
```

12. Add the script to the routes with the file name `web.php`. Add as below image

```
Route::get('/user/ubah/{id}', [UserController::class, 'ubah']);
```

13. Add a script to the controller with the file name `UserController.php`. Add the script in the class and create a new method with the change name and put it under the `tambah_simpan` method as shown below

```
public function ubah($id)
{
    $user = UserModel::find($id);
    return view('user_ubah', ['data' => $user]);
}
```




14. Save the program code Step 11 through 13. Then run it on the *browser* and click the **"Change"** link, observe what happens and give an explanation in the report
15. Add *the script* to the *routes* with the file name `web.php`. Add as below image

```
Route::put('/user/ubah_simpan/{id}', [UserController::class, 'ubah_simpan']);
```

16. Add *a script* to the controller with the file name `UserController.php`. Add *the script* in the class and create a new method with the name `ubah_simpan` and put it under the `change` method as below screenshot

```
public function ubah_simpan($id, Request $request)
{
    $user = UserModel::find($id);

    $user->username = $request->username;
    $user->nama = $request->nama;
    $user->password = Hash::make($request->password);
    $user->level_id = $request->level_id;

    $user->save();

    return redirect('/user');
}
```

17. Save the program code for Steps 15 and 16. Then run the link `localhost:8000/user/change/1` or `localhost/PWL_POS/public/user/change/1` in a *browser* and change the input form and click the change button, then observe what happens and give an explanation in the report
18. Here is the delete step. Add *the script* to the *routes* with the file name `web.php`. Add as below image

```
Route::get('/user/hapus/{id}', [UserController::class, 'hapus']);
```

19. Add *a script* to the controller with the file name `UserController.php`. Add *the script* in the class and create a new method with the name `delete` and put it under the `ubah_simpan` method as below screenshot

```
public function hapus($id)
{
    $user = UserModel::find($id);
    $user->delete();

    return redirect('/user');
}
```

20. Save the program code for Steps 18 and 19. Then run it on the *browser* and click the delete button, then observe what happens and give an explanation in the report
21. Report the results of this Practicum-2.6 and *commit* the changes to *git*.



Practicum 2.7 – Relationships

One to One

A one-to-one relationship is a very basic type of database relationship. For example, a `Usermodel` might be associated with a single `Levelmodel` model. To define this relationship, we'll place the `Levelmodel` method on the `Usermodel` model. The `Levelmodel` method should call the `hasOne` method and return the result. This method `hasOne` is available for your model through the model base class `Illuminate\Database\Eloquent\Model`:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasOne;

You, 1 second ago | 1 author (You)
class UserModel extends Model
{
    public function level(): HasOne
    {
        return $this->hasOne(LevelModel::class);
    }
}
```

Defining the Opposite of a *One-to-one Relationship*

So, we can access the `LevelModel` model from our `UserModel` model. Next, let's define the relationship in the `LevelModel` model that allows us to access the user. We can define the opposite of a `hasOne` relationship using the `belongsTo` method:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class LevelModel extends Model
{
    public function user(): BelongsTo
    {
        return $this->belongsTo(UserModel::class);
    }
}
```



One to Many

A one-to-many relationship is used to define a relationship in which one model is the parent of one or more child models. For example, 1 category may have an unlimited number of items. Like all other Eloquent relationships, the one-to-many relationship is defined by defining a method on your Eloquent model:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class KategoriModel extends Model
{
    public function barang(): HasMany
    {
        return $this->hasMany(BarangModel::class, 'barang_id', 'barang_id');
    }
}
```

One to Many (Inverse) / Belongs To

Now that we can access all the goods, let's define the relationship so that the goods can access their parent categories. To determine the inverse of a `hasMany` relationship, specify the relationship method on the child model that calls `belongsTo` :

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class BarangModel extends Model
{
    public function kategori(): BelongsTo
    {
        return $this->belongsTo(KategoriModel::class, 'kategori_id', 'kategori_id');
    }
}
```

1. Open the model file on `UserModel.php` and add the scrip to look like below



```
class UserModel extends Model
{
    use HasFactory;

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
    /**
     * The attributes that are mass assignable.
     *
     * @var array
     */
    protected $fillable = ['level_id', 'username', 'nama', 'password'];

    public function level(): BelongsTo
    {
        return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
    }
}
```

2. Open the controller file on [the UserController.php](#) and change the script method to something like below

```
public function index()
{
    $user = UserModel::with('level')->get();
    dd($user);
}
```

3. Save the program code Step 2. Then run the link in the *browser*, then observe what happens and give an explanation in the report
4. Open the controller file on [the UserController.php](#) and change the script method to something like below

```
public function index()
{
    $user = UserModel::with('level')->get();
    return view('user', ['data' => $user]);
}
```

5. Open the file view on [user.blade.php](#) and change *the script* to something like below

```
<body>
<h1>Data User</h1>
<a href="/user/tambah">+ Tambah User</a>
<table border="1" cellpadding="2" cellspacing="0">
    <tr>
        <td>ID</td>
        <td>Username</td>
        <td>Nama</td>
        <td>ID Level Pengguna</td>
        <td>Kode Level</td>
        <td>Nama Level</td>
        <td>Aksi</td>
    </tr>
    @foreach ($data as $d)
        <tr>
            <td>{{ $d->user_id }}</td>
            <td>{{ $d->username }}</td>
            <td>{{ $d->nama }}</td>
            <td>{{ $d->level_id }}</td>
            <td>{{ $d->level->level_kode }}</td>
            <td>{{ $d->level->level_nama }}</td>
            <td><a href="/user/ubah/{{ $d->user_id }}">Ubah</a> | <a href="/user/hapus/{{ $d->user_id }}">Hapus</a></td>
        </tr>
    @endforeach
</table>
</body>
```



6. Save the program code Step 4 and 5. Then run the link in the *browser*, then observe what happens and give an explanation in the report
7. Report the results of this Practicum-2.7 and *commit* the changes to *git*.

*That's all, and happy learning ****